# プログラミングの基本

## 変数・代入・出力



- **変数宣言，代入**　`a = 10`, `a = b`
  - Cのような変数宣言は不要

```
int a = 10;
int b = 20;
a = b;
```

**Cの記法**

- **出力**　`表示する(a)`
  - 変数 a が記憶している**値**を画面上に表示
  - 「a」という**文字列**を表示したいときは，`print("a")`
  - 末尾に改行が入る
    - 改行を入れずに空白を入れる：`末尾に空白を入れて表示する(a)`

```
printf("a¥n");
printf("%d¥n", a);
```

## 変数・代入・出力



- **サンプルコード**

```
a = 10
b = 20
a = b
表示する(a)
```

- **実行結果**
  - **20**

```
#include <stdio.h>
int main(void) {
    int a = 10;
    int b = 20;
    a = b;
    printf("%d¥n", a);
    return 0;
}
```

## 四則演算の書き方



- **足し算**：a + b
- **引き算**：a - b
- **掛け算**：a * b
- **割り算**：a / b
- **その他の計算**
  - a ÷ b の余り：　a % b
  - a ÷ b の整数部分：　a ÷ b
  - a の b 乗：　a ** b

## 条件分岐 if

7

- 処理を**条件**により選択する.
  ⇒ 条件が**真(YES)**のときは処理群1，**偽(NO)** のときは処理群2を実行
- ｜，└ で処理群を区別（学習ツールで入力可）

```
もし 条件式 ならば:
｜ 処理群
そうでなくもし 条件式 ならば:
｜ 処理群
そうでなければ:
└ 処理群
```

```
if (条件式) {
    処理群
} else if (条件式) {
    処理群
} else {
    処理群
}
```

## 条件式の書き方

8

- x と y の値が等しい　=　: x == y
- x と y の値が等しくない　≠　: x != y
- x が y より小さい　<　: x < y
- x が y 以下　≦　: x <= y
- x が y より大きい　>　: x > y
- x が y 以上　≧　: x >= y

## 条件分岐 if

9

- サンプルコード

```
t = 16
もし t < 12 ならば:
｜ 表示する("おはよう")
そうでなくもし t <= 17 ならば:
｜ 表示する("こんにちは")
そうでなければ:
└ 表示する("こんばんは")
```

```
#include <stdio.h>
int main(void) {
    int t = 16;
    if (t < 12) {
        printf("おはよう¥n");
    } else if (t <= 17) {
        printf("こんにちは¥n");
    } else {
        printf("こんばんは¥n");
    }
    return 0;
}
```

## 繰り返し while

10

- 処理を**繰り返し**実行する。
  ⇒ 条件が**真(YES)**の間，**処理群を繰り返し実行**し，
  **偽(NO)** になるとループ終端の下にある次の処理を実行する.
- 繰り返し条件を記述するために，
  **カウンタの役割を持つ変数を用意**することが多い.
  - 処理群の最後でカウントアップするなど，
    **処理群の中で値を変更**しないと無限ループしてしまう

```
条件式 の間繰り返す:
└ 処理群
次の処理
```

```
while (条件式) {
    処理群
}
次の処理
```

## 繰り返し while

11

- サンプルコード

```
n = 3
a = 0
i = 0
i < n の間繰り返す:
│ a = a + 2
└ i = i + 1
表示する(a)
```

```c
#include <stdio.h>
int main(void) {
    int n = 3, a = 0, i = 0;
    while (i < n) {
        a = a + 2;
        i = i + 1;
    }
    printf("%d¥n",a);
    return 0;
}
```

## 繰り返し for

12

- 処理を**繰り返し**実行する。

```
i を a から b まで c ずつ増やしながら繰り返す:
└ 処理群
次の処理
```

  - 処理群を実行するごとに変数の値が
    **a ≦ 変数の値 ≦ b** の範囲内で c ずつ変化する
  - 減らしながら繰り返すもある

```
for (前処理; 条件式; 後処理) {
    処理群
}
次の処理
```

## 繰り返し for

13

- サンプルコード

```
a = 0
i を 0 から 2 まで 1 ずつ増やしながら繰り返す:
└ a = a + 2
表示する(a)
```

```c
#include <stdio.h>
int main(void) {
    int i, a = 0;
    for (i=0; i<3; ++i) {
        a = a + 2;
    }
    printf("%d¥n", a);
    return 0;
}
```

## 配列（リスト）

14

- リスト　`List = [1, 2, 3]`　　`int list[3] = {1, 2, 3};`
  - Cのような変数宣言は不要
  - **変数名の先頭は大文字**にする
  - リストを空で初期化したい場合　`List = []`
- リストの操作
  - リスト List の要素数を調べる　`要素数(List)`
  - リスト List に要素を追加する　`List.追加(追加する値)`
- リストの要素を表示（添字は0番目から開始）
  - リスト List の i 番目の要素を表示　`表示する(List[i])`
  - リスト list の全ての要素を表示　`表示する(List)`
    - 出力例：`[1, 2, 3]`

## 配列（リスト）

15

- サンプルコード

```
List = [1, 2, 3, 4, 5]
sum = 0
i を 0 から 要素数(List)-1 まで 1 ずつ増やしながら繰り返す:
└ sum = sum + List[i]
表示する(sum)
```

```
#include <stdio.h>
int main(void) {
    int list[5] = {1, 2, 3, 4, 5};
    int i, sum = 0, len = 5;
    for (i=0; i<len; ++i) {
        sum = sum + list[i];
    }
    printf("%d¥n", sum);
}
```

## 関数

16

- 関数宣言・return文

```
関数 関数名(引数, …) を定義する:
│ 処理群
└ 値 返り値 を返す
```

- 関数は呼び出す前に定義する
- Pythonでは，関数の引数としてリストを渡すことができる

```
返り値の型 関数名(引数の型 引数, …) {
    処理群
    return 返り値;
}
```

## 関数

17

- サンプルコード

```
関数 goukei(list) を定義する:
│ sum = 0
│ i を 0 から 要素数(List)-1 まで 1 ずつ増やしながら繰り返す:
│ └ sum = sum + List[i]
└ 値 sum を返す

List = [12, 13, 14]
表示する(goukei(List))
```

```
#include <stdio.h>

int sum(int *a, int len) {
    int sum = 0, i;
    for (i=0; i<len; ++i) {
        sum = sum + a[i];
    }
    return sum;
}

int main(void) {
    int a[3] = {12, 13, 14};
    printf("%d¥n", sum(a, 3));
}
```

## 学習ツールによるプログラムの実行

18

- 学習ツールで以下のプログラムを打ち込み，実行してください
  - Runボタンを押す or Ctrl+Enter

```
関数 max_i(List) を定義する:
│ max_i = 0
│ i を 1 から 要素数(List)-1 まで 1 ずつ増やしながら繰り返す:
│ │ もし List[i] > List[max_i] ならば:
│ └ └ max_i = i
└ 値 max_i を返す

List = [34, 12, 45, 98, 59]
表示する(List[max_i(List)])
```

│ : Tab
└ : Shift + Tab